Capstone Final Project (OpenMarket Corp. E-Market)

Software Architecture 1

Jala University

1. Identify architecture characteristics.

Architecture Characteristics Worksheet

System/Project: **OpenMarket Corp. • E-Market Platform** Domain/Quantum: **E-Commerce**

Architect/Team: **Jalasoft Jala-U GA SE G3** Date: 02/25/2026 Next Review: 08/25/2026

Top 3	Driving Characteristics	Implicit Characteristics
[x]	Security	Maintainability
[ ]	Interoperability	Deployability
[ ]	Data Consistency	Testability
[x]	Availability	
[x]	Elasticity	
[ ]	Responsiveness	
[ ]	Extensibility	

Others Considered
Fault Tolerance
Configurability
Scalability

Instructions

- Identify no more than 7 driving characteristics.
- Pick the top 3 characteristics (in any order).
- Implicit characteristics can become driving characteristics if they are *critical* concerns.
- Add additional characteristics identified that weren't deemed as important as the list of 7 to the *Others Considered* list.
- Definitions are on the following pages

Common Architecture Characteristics

performance	data integrity	deployability
responsiveness	data consistency	testability
availability	adaptability	abstraction
fault tolerance	extensibility	workflow
scalability	interoperability	configurability
elasticity	concurrency	recoverability

Composite Architecture Characteristics

agility → maintainability, testability, deployability

reliability → availability, testability, data integrity, data consistency, fault tolerance

a, b denotes characteristics that are related; some systems only need one of these, other systems may need both

TOP 3 Driving Characteristics

Elasticity:

Related to Requirement 3.1. Essential for supporting massive traffic spikes without impacting the user experience.

Availability:

Related to Requirement 3.3. Critical for preventing prolonged downtime during peak sales seasons.

Security:

Related to Requirement 3.4. Requires the protection of financial and personal data, strictly complying with the PCI-DSS standard.

Other Driving Characteristics Considered

Interoperability:

Related to Requirements 1.1, 1.4, and 2.4. Necessary for integrating suppliers via CSV/REST, payment gateways, and shipping carriers.

Extensibility:

Requirement 4.4 anticipates the addition of new features (e.g., subscriptions, loyalty programs) without fundamentally altering the base system.

Responsiveness:

Requirement 2.1 demands an agile UI where the customer does not experience excessive delays when browsing or filtering.

Data Consistency:

Requirements 1.5 and 2.5 mandate accurate inventory tracking and require immediate stock confirmation upon a sale.

Other considered

Fault Tolerance

Related to the heavy reliance on external integrations (Requirements 1.4, 1.5, and 2.4). If a third-party payment or logistics API goes down, the system must degrade gracefully rather than crashing the entire purchase flow.

Configurability

Related to Requirement 3.2 (Multiple time zones and languages). The architecture must dynamically adapt to different currencies, regional taxes, and date formats without requiring hard-coded changes for each new country.

Scalability

Considered due to the need to handle an increasing number of concurrent users and a growing catalog of products over time (Req 1.1 and 3.1). However, Elasticity was prioritized as the driving characteristic because the system must not only scale up during traffic spikes but also scale down when demand decreases to optimize operational costs.

Implicit Characteristics

Maintainability:

With multiple developers and new features planned for the future, the code must be clean, modular, and easy to maintain. It must be properly documented.

Deployability:

Given the "frequent updates" and "multiple suppliers", the system must be able to be updated and integrated continuously (CI/CD) without taking the store offline. Best practices must be followed.

Testability:

When handling third-party payments and complex inventories, critical components must be easily subjected to automated testing. There must be unit tests covering edge cases, integration tests (to isolate communication with gateways and carriers), and E2E tests (to validate the entire checkout flow).

2.Select the architectural style.

Architecture Styles Worksheet

System/Project: **OpenMarket Corp. • E-Market Platform**

Architect/Team: **Jalasoft Jala-U GA SE G3** Date: 02/25/2026

Selected Architecture(s):

	layered	modular monolith	microkernel	microservices	service-based	service-oriented	event-driven	space-based
Selected	☐	☐	☐	☒	☐	☐	☐	☐
partitioning	technical	domain	domain	domain	domain	technical	technical	technical
cost	$	$	$	$$$$$	$$	$$$$	$$$	$$$$
maintainability	★	★★	★★★	★★★★★	★★★★	★	★★★	★★★
testability	★★	★★	★★★	★★★★★	★★★★	★	★★	★
deployability	★	★★	★★★	★★★★★	★★★★	★	★★★	★★★
simplicity	★★★★★	★★★★★	★★★★	★	★★★	★	★★	★
scalability	★	★	★	★★★★★	★★★	★★★	★★★★	★★★★★
elasticity	★	★	★	★★★★	★★	★★★	★★★	★★★★★
responsiveness	★★★	★★★	★★★	★★	★★★	★★	★★★★★	★★★★★
fault-tolerance	★	★	★	★★★★★	★★★★	★★★	★★★★★	★★★
evolvability	★	★	★★★	★★★★★	★★★★	★	★★★★★	★★★
abstraction	★	★	★★★	★	★	★★★★★	★★★★	★
interoperability	★	★	★★★	★★★	★★	★★★★★	★★★	★★

Architecture Styles Worksheet

To determine the most appropriate architectural structure for OpenMarket Corp E-Market, three architectural styles were evaluated based on the previously identified driving characteristics: Elasticity, Availability, and Security.

Architectural Style	Advantages	Disadvantages	Alignment with Driving Characteristics
Monolithic Architecture	Simple development and deployment, easier initial setup.	Limited scalability, high coupling, risky deployments affecting the entire system.	Poor alignment with Elasticity and Availability requirements.
Microservices Architecture	Independent scalability, fault isolation, continuous deployment capability, service autonomy.	Increased operational complexity and infrastructure management.	Strong alignment with Elasticity, Availability, and Security.
Event-Driven Architecture	Loose coupling, asynchronous communication, high responsiveness.	Complex debugging and data consistency challenges.	Good alignment with Extensibility and Responsiveness but higher implementation complexity.

Architectural Style Selection

After comparing the alternatives, the selected architectural approach is:
Microservices Architecture complemented with Event-Driven communication.

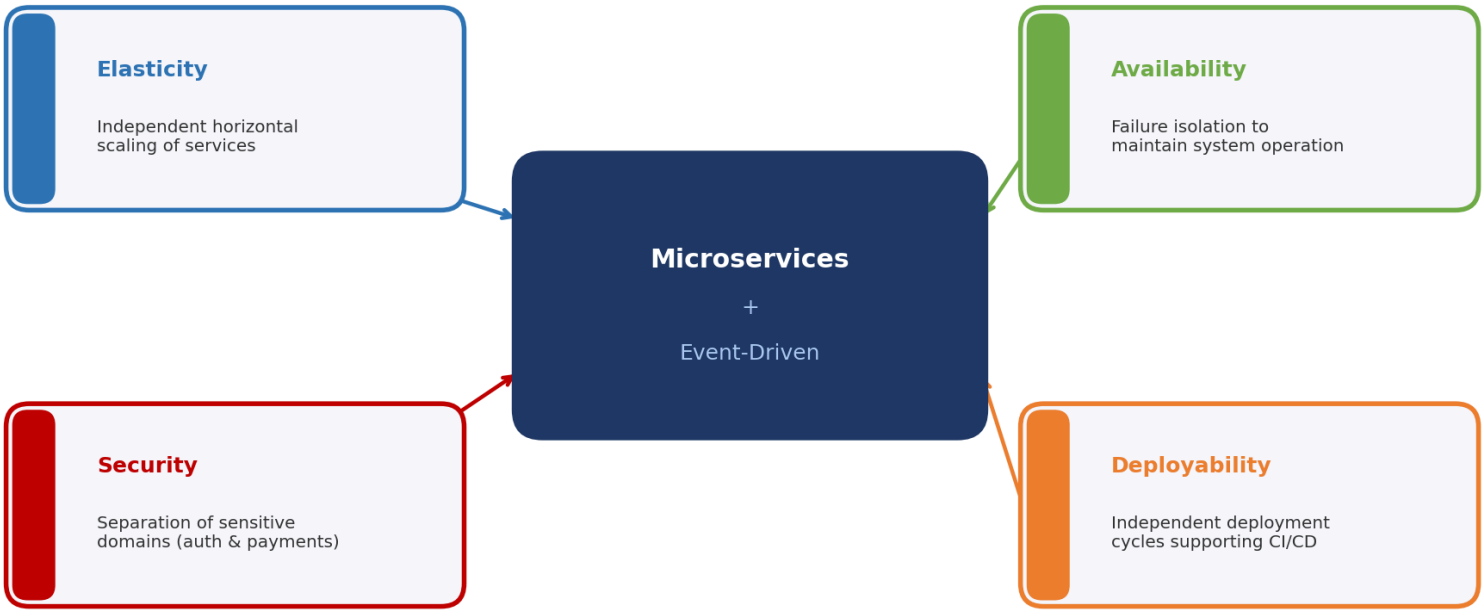
Justification

The OpenMarket platform must support large traffic spikes, multiple external integrations, and continuous evolution of features without impacting system stability.
A microservices architecture enables:

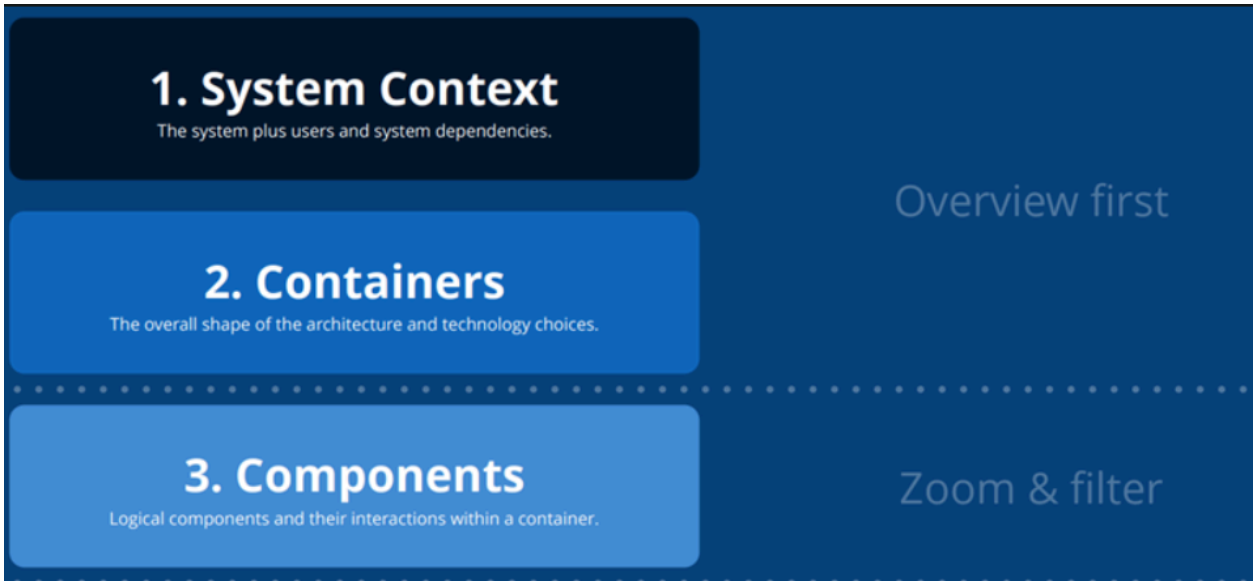
- Independent horizontal scaling of services (Elasticity).
- Failure isolation to maintain system operation (Availability).
- Separation of sensitive domains such as authentication and payments (Security).
- Independent deployment cycles supporting CI/CD pipelines (Deployability).

Event-driven communication will be used as a complementary approach for asynchronous processes such as notifications, inventory updates, and post-payment operations.

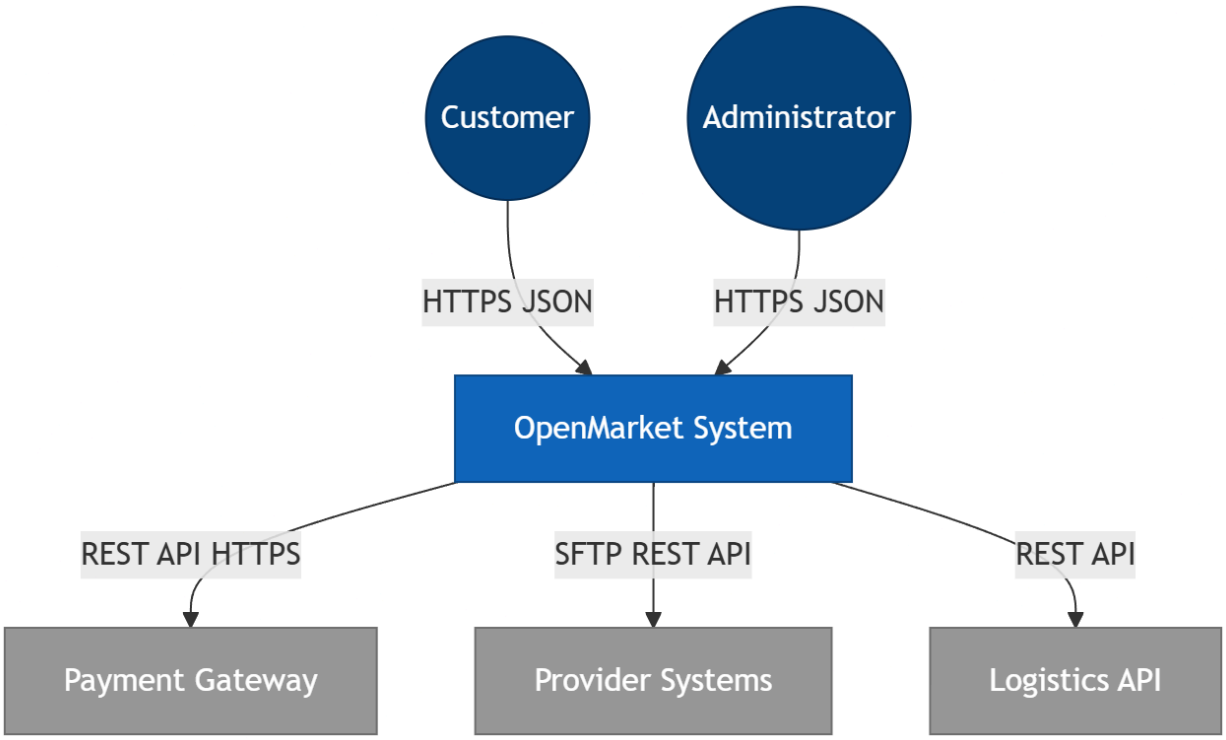
Why Microservices + Event-Driven?



3.Architectural representation (using the C4 model).



C1



Technical Annotations:

Customer/Admin to System:

Communication via HTTPS ensuring secure data transfer. Data format is JSON for API consumption or HTML for web rendering.

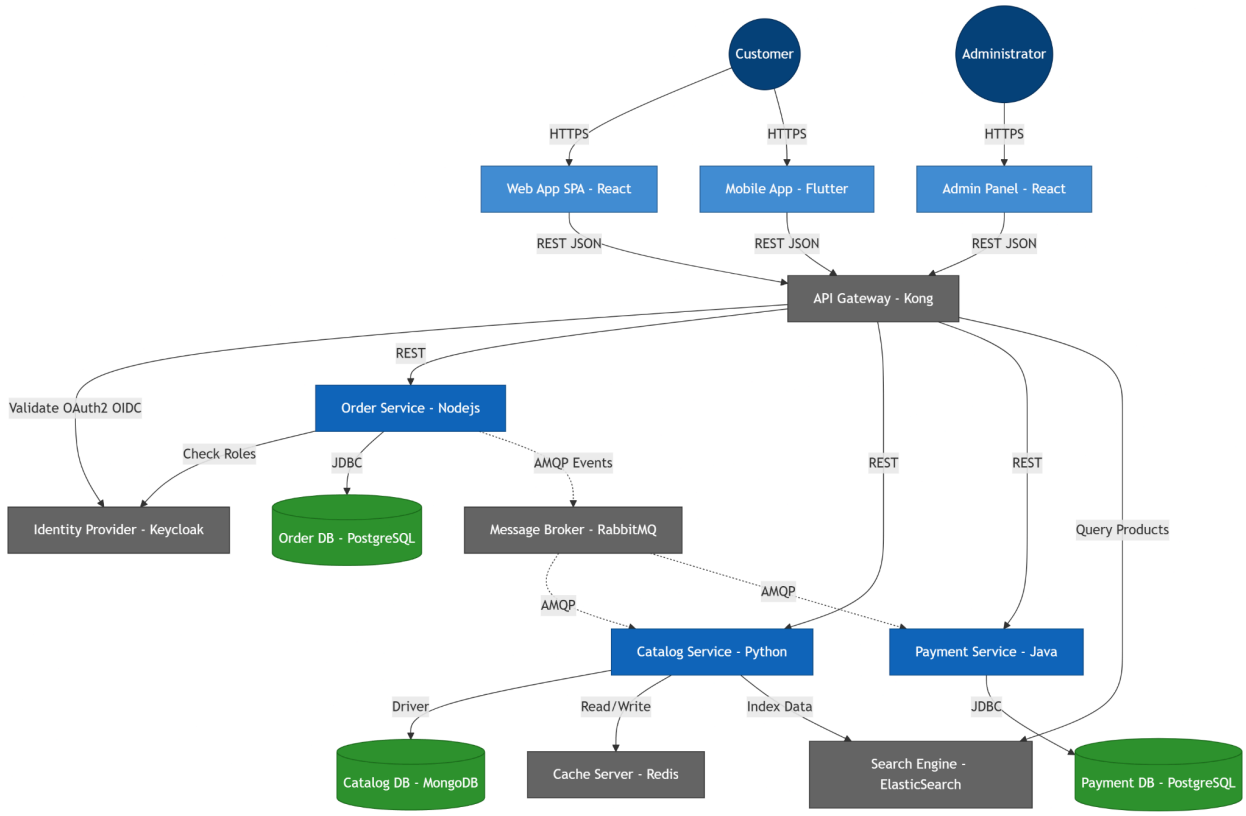
System to Payment Gateway:

Synchronous integration via REST API (e.g., Stripe/PayPal). Critical for Security (PCI-DSS) compliance as per Román's Req 3.4.

System to Provider Systems:

Hybrid integration. SFTP for batch CSV file processing (Req 1.1) and REST for real-time catalog updates, ensuring Interoperability.

C2



Technical Annotations:

Identity Provider (Keycloak):

Centralizes Security (Req 3.4). Handles user authentication and issues JWT tokens. Uses OIDC (OpenID Connect) protocol.

Cache Server (Redis):

In-memory data store. Drastically improves Responsiveness (Req 2.1) by caching product listings and prices. Uses TCP protocol.

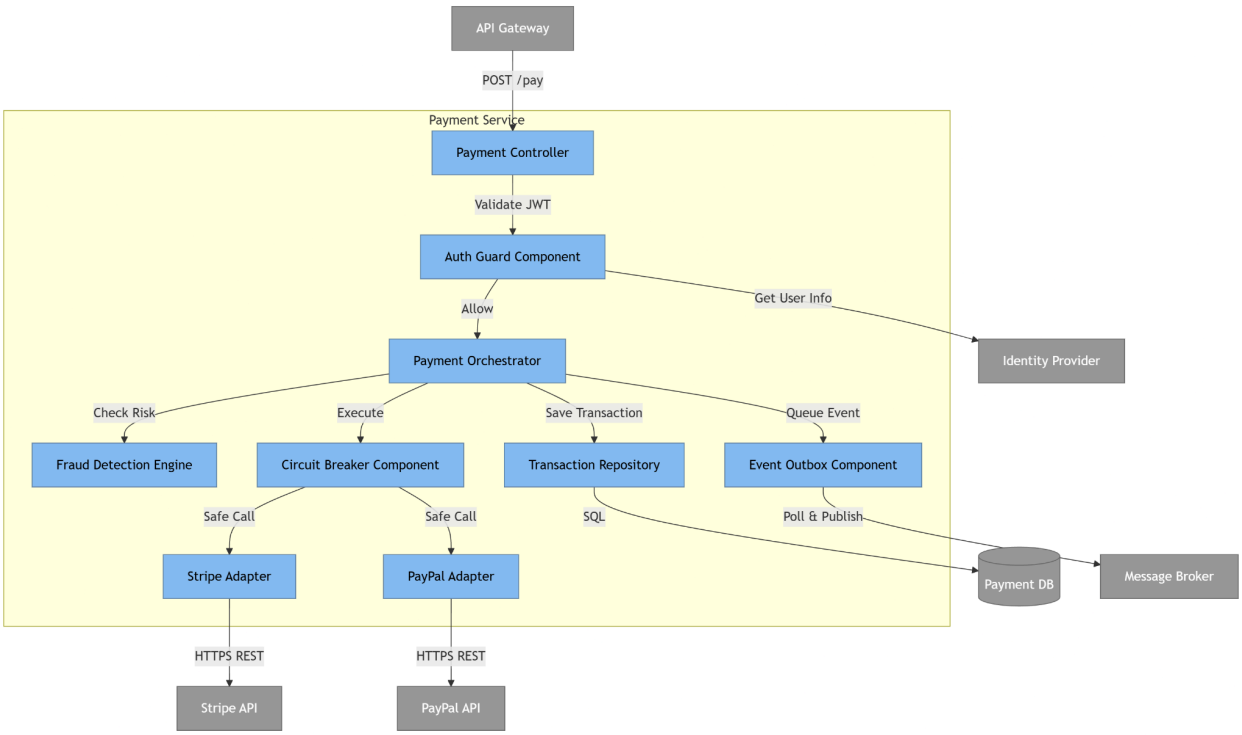
Search Engine (ElasticSearch):

Dedicated indexer. Allows complex filtering and text search without overloading the main database.

Database per Service:

Maintains data isolation (Consistency).

C3



Technical Annotations (level 3) for payment service:

Circuit Breaker Component:

Implements the Circuit Breaker Pattern (e.g., using Resilience4j). If external payment APIs are down, it fails fast, protecting the system from thread blocking, ensuring high Availability (Req 3.3).

Fraud Detection Engine:

Analyzes transaction metadata (IP location, amount) before processing. Increases Security beyond just authentication.

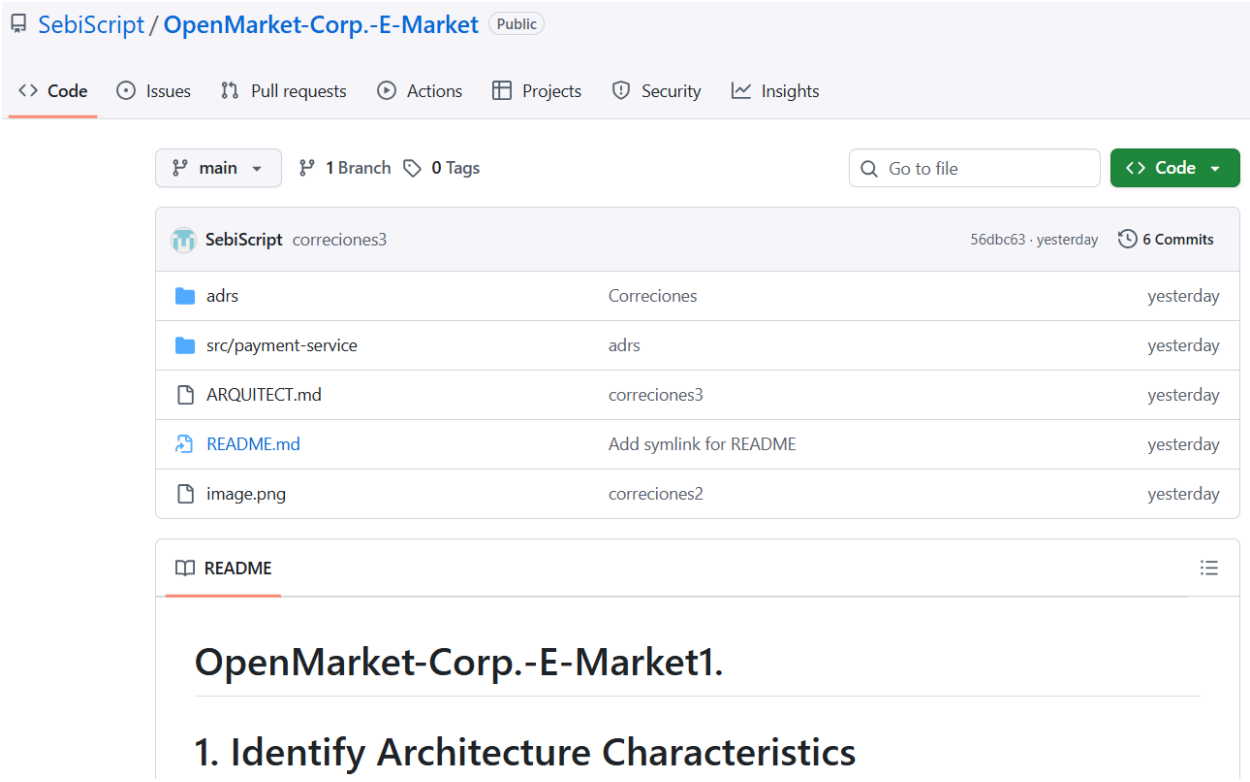
Event Outbox Component:

Ensures that a database transaction and a message broker publication happen atomically. Guarantees Data Consistency (Req 2.5) — "The payment was saved AND the notification was sent."

Auth Guard:

Communicates with the Identity Provider via REST to validate tokens on every critical request.

4. Code Repository

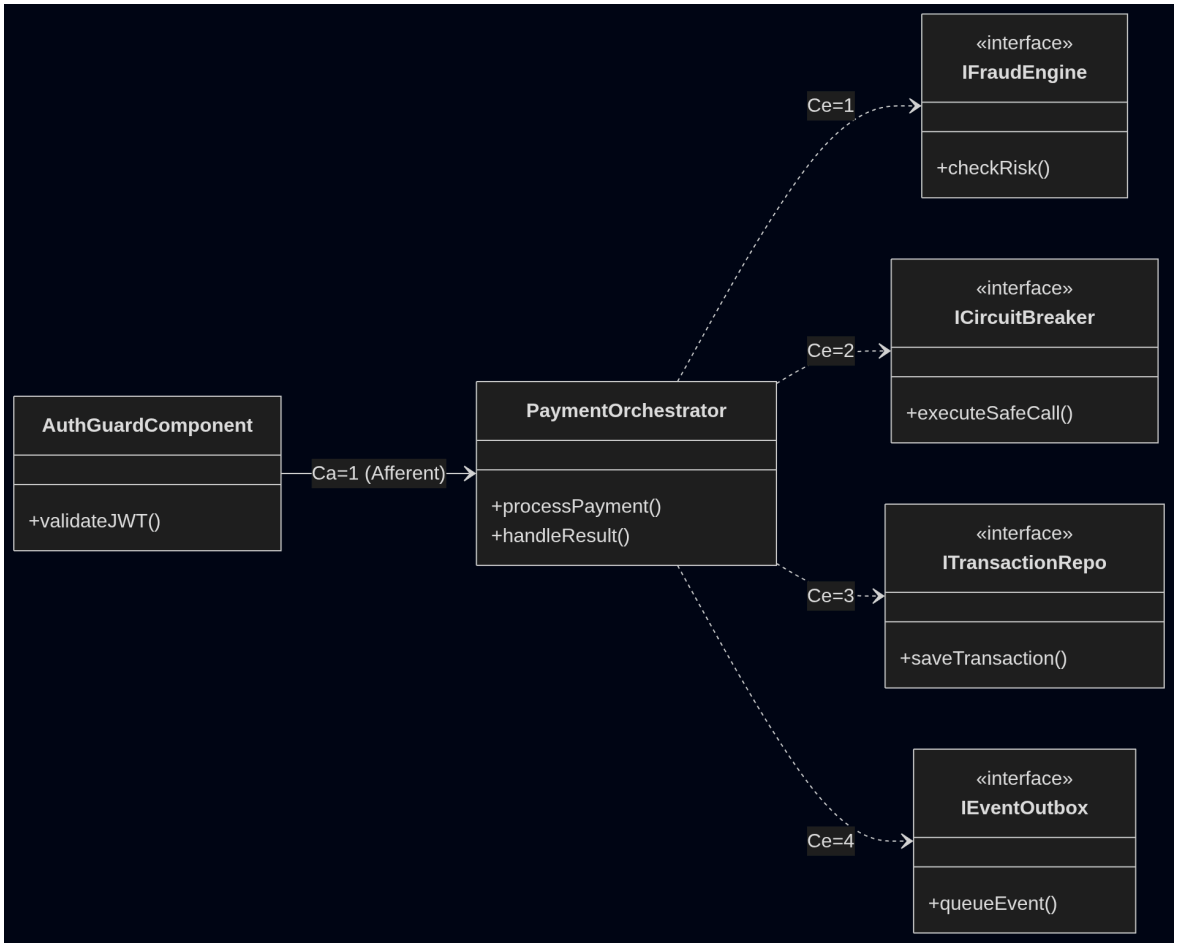


Weblink (URL):

https://github.com/SebiScript/OpenMarket-Corp.-E-Market

5. Class Diagrams

For the solution to this task, the Payment Service component will be used, as it is the central component that contains the business logic and also coordinates both internal and external dependencies.
The diagram created in the previous step represents the internal structure and the relationships of the orchestrator with its supporting services, which are:



Class / Interface	Basic Functions	Responsibility
PaymentOrchestrator	processPayment(), handleResult()	The core engine that orchestrates the logic between fraud checks,

		persistence, and external gateways.
IFraudEngine	checkRisk()	Decouples the orchestrator from specific fraud detection logic or third-party providers.
ICircuitBreaker	executeSafeCall()	Manages resilience by preventing thread exhaustion during external API (Stripe/PayPal) outages.
ITransactionRepo	saveTransaction()	Abstracts the data access layer, allowing the orchestrator to remain agnostic of the underlying database (SQL/NoSQL).
IEventOutbox	queueEvent()	Ensures atomic consistency by queuing events to be published to the Message Broker after a successful transaction.

To determine the soundness of the design, we will calculate the **Distance from the Main Sequence**, which will be represented by the letter *D*.

Abstraction (A)

To calculate abstraction, we must determine the ratio between the number of abstract classes (or interfaces) and the total number of classes in the component.

N_a *Abstract elements:*

4 IFraudEngine, ICircuitBreake, ITransactionRepo, IEventOutbox

N_c *Total elements:*

5 the orchestrator and the 4 interfaces

$$A = \frac{N_a}{N_c} = \frac{4}{5} = 0.8$$

Instability (I)

To obtain the instability value, it is calculated based on the dependencies that enter and leave the component.

C_a *(Afferent Coupling):*

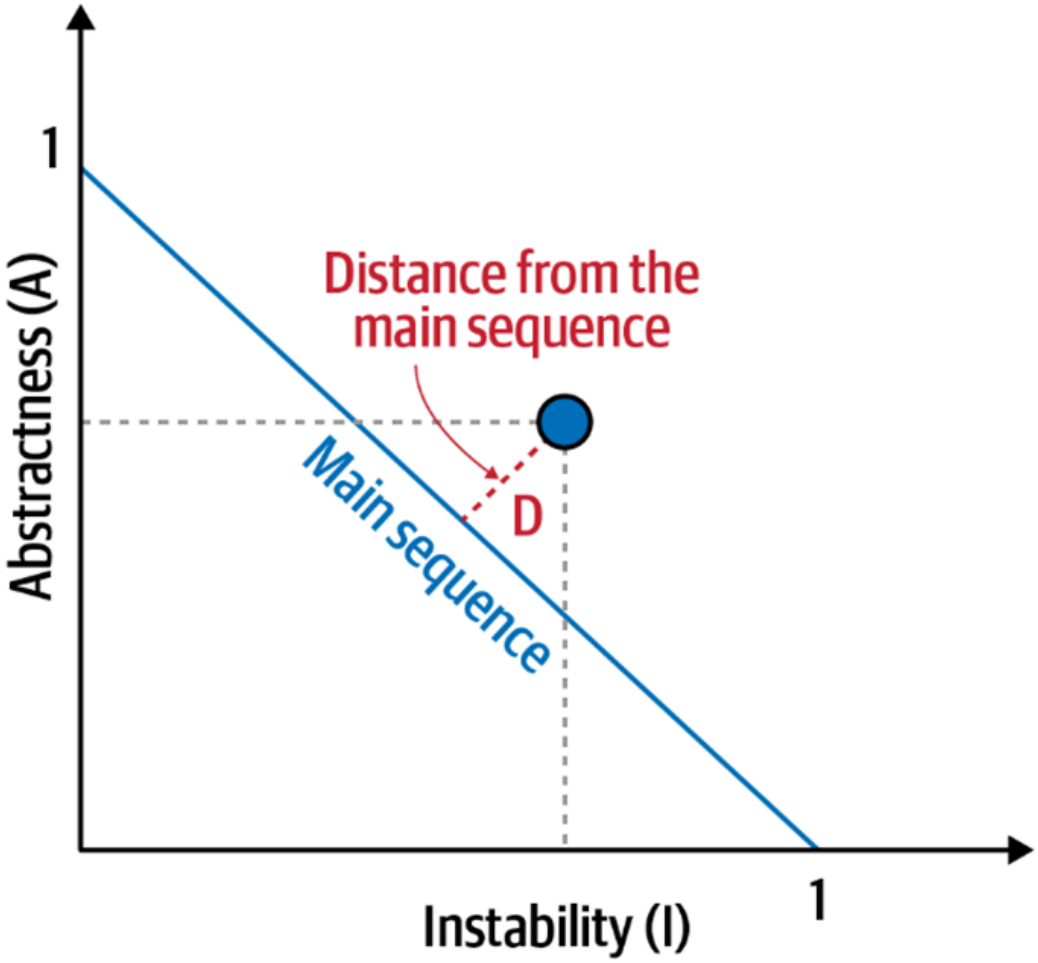
1. The Auth Guard Component depends on the orchestrator to initiate the payment flow.

C_e *(Efferent Coupling):*

4. The orchestrator depends on 4 different abstractions to fulfill its business logic.

$$I = \frac{C_e}{C_a + C_e} = \frac{4}{1+4} = 0.8$$

6.Distance from the main sequence



To obtain the Main Sequence value, which is the objective, we take the absolute value of the sum of Abstraction and Instability minus 1.

$$D = |A + I - 1|$$

$$D = |0.8 + 0.8 - 1|$$

$$D = |1.6 - 1| = 0.6$$

After calculating the metrics, a distance of 0.6 was obtained. In software architecture (Clean Architecture), an ideal component lies on the Main Sequence $D \approx 0$, achieving a balance between its extensibility and its stability.

The value D = 0.6 indicates that the component is far from the main sequence. It shows a high level of abstraction together with high instability, placing it near the zone of uselessness. This suggests that the component has many interfaces but strongly depends on other external modules, reducing its structural stability.
Although the design is flexible, its high efferent coupling may affect reusability and robustness in the face of environmental changes.
In conclusion, the result is regular-to-low, and it is recommended to review whether all interfaces, such as Fraud Engine or Event Outbox, are strictly necessary or whether the design can be simplified to move closer to the main sequence component.

7. Self-assessment on team participation:

- Alejandro: 5/5
- Cristian: 5/5
- Román: 5/5
- Said: 5/5
- Sebastián: 5/5